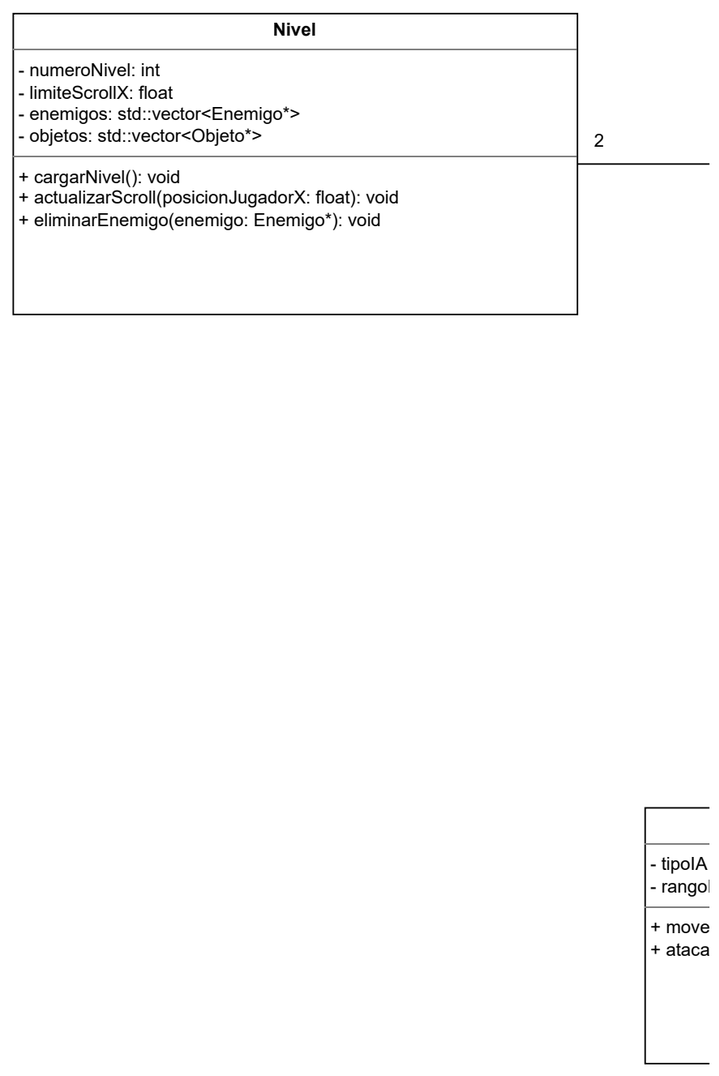
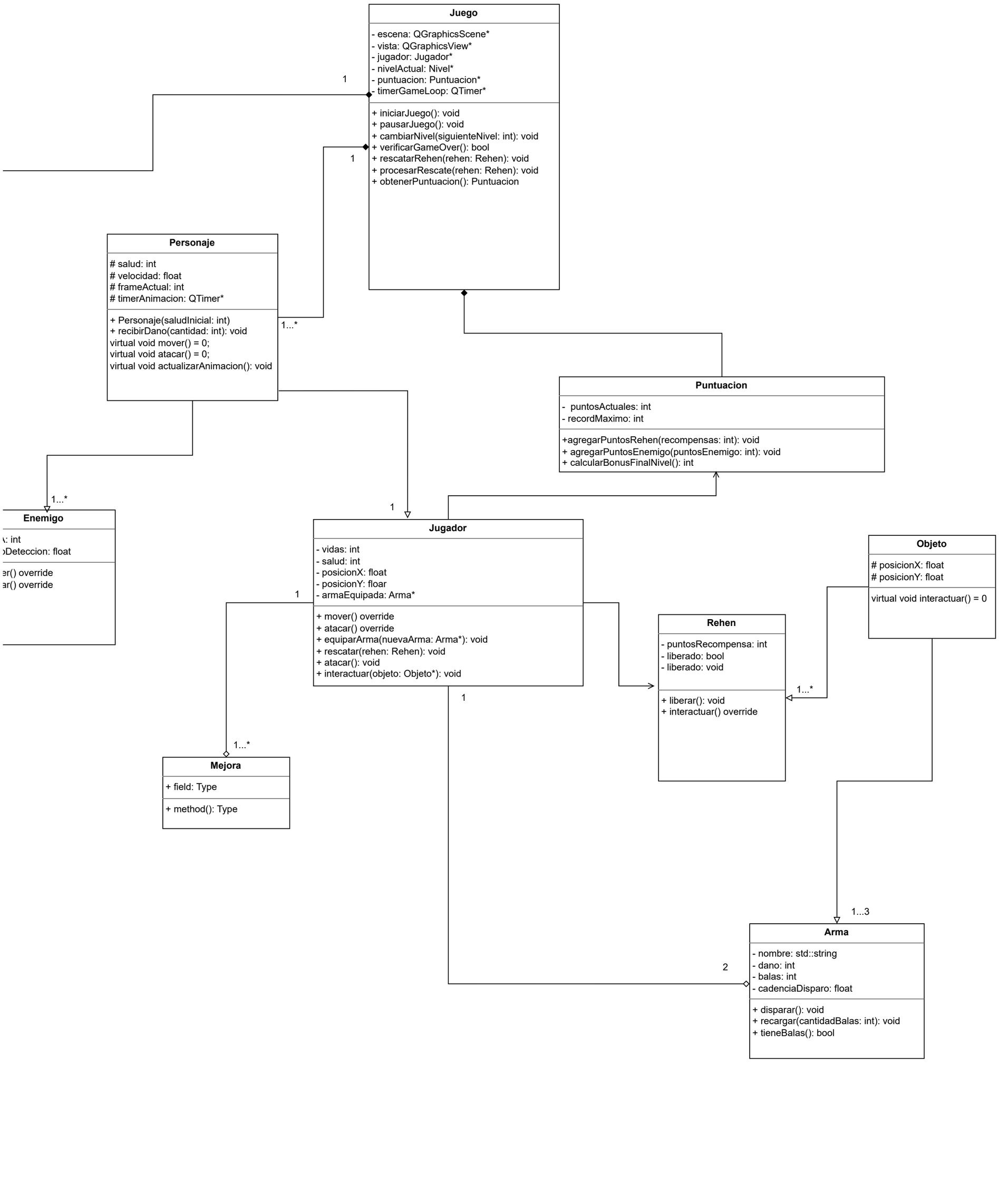

Nivel
- numeroNivel: int
- limiteScrollX: float
- enemigos: std::vector<Enemigo*>
- objetos: std::vector<Objeto*>
+ cargarNivel(): void
+ actualizarScroll(posicionJugadorX: float): void
+ eliminarEnemigo(enemigo: Enemigo*): void
2
- tipoIA
- rango
+ move
+ ataca


Juego
- escena: QGraphicsScene*
- vista: QGraphicsView*
- jugador: Jugador*
- nivelActual: Nivel*
- puntuacion: Puntuacion*
- timerGameLoop: QTimer*
+ iniciarJuego(): void
+ pausarJuego(): void
+ cambiarNivel(siguienteNivel: int): void
+ verificarGameOver(): bool
+ rescatarRehen(rehen: Rehen): void
+ procesarRescate(rehen: Rehen): void
+ obtenerPuntuacion(): Puntuacion
1
1
Personaje
salud: int
velocidad: float
frameActual: int
timerAnimacion: QTimer*
+ Personaje(saludInicial: int)
+ recibirDano(cantidad: int): void
virtual void mover() = 0;
virtual void atacar() = 0;
virtual void actualizarAnimacion(): void
1...*
Puntuacion
- puntosActuales: int
- recordMaximo: int
+agregarPuntosRehen(recompensas: int): void
+ agregarPuntosEnemigo(puntosEnemigo: int): void
+ calcularBonusFinalNivel(): int
1...*
Enemigo
: int
Deteccion: float
er() override
ar() override
1
Jugador
- vidas: int
- salud: int
- posicionX: float
- posicionY: floar
- armaEquipada: Arma*
+ mover() override
+ atacar() override
+ equiparArma(nuevaArma: Arma*): void
+ rescatar(rehen: Rehen): void
+ atacar(): void
+ interactuar(objeto: Objeto*): void
Objeto
posicionX: float
posicionY: float
virtual void interactuar() = 0
1
Rehen
- puntosRecompensa: int
- liberado: bool
- liberado: void
+ liberar(): void
+ interactuar() override
1...*
1...*
Mejora
+ field: Type
+ method(): Type
1
1...3
2
Arma
- nombre: std::string
- dano: int
- balas: int
- cadenciaDisparo: float
+ disparar(): void
+ recargar(cantidadBalas: int): void
+ tieneBalas(): bool